

以 **XLS-R+SLS 原项目** 为主，不整体迁移 Multi-level SSL 项目；只参考并移植其中的门控思想。

按“统计感知 SLS → SwiGLU → 时间注意力”顺序逐步加入，每一步都保留开关和消融。

原 SLS 仓库的训练、评估、数据处理和 fairseq 环境已经完整，官方使用 Python 3.7、PyTorch 1.12.1、XLS-R 300M，训练入口为 `main.py`，模型主体为 `model.py`。(GitHub)

注意以下代码应该是轻量版的，可以让codex根据2个项目情况自己写

一、最终模型结构

```
原始音频
↓
RawBoost
↓
XLS-R 24层隐藏特征
↓
Mean+Std统计感知SLS
↓
多层特征加权求和
↓
SwiGLU特征门控
↓
时间注意力池化
↓
全连接二分类
```

三个改进（全在后端）分别是：

1. Mean+Std统计感知SLS

原 SLS 只用均值计算层权重，改为同时使用均值和标准差。

2. SwiGLU特征门控

SLS选择重要层后，再选择融合特征中的重要维度。

3. 时间注意力池化

替代简单的固定池化，让模型关注含有伪造痕迹的关键帧。

这三部分与两篇论文的核心方向一致：充分利用 XLS-R 多层特征，并进行动态门控。

二、项目目录安排

建议目录：

```
workspace/  
├── SLSforASVspoof-2021-DF/  
└── dissimilarity_deepfake_detection/
```

执行：

```
mkdir workspace  
cd workspace  
  
git clone https://github.com/QiShanZhang/SLSforASVspoof-2021-DF.git  
  
git clone https://github.com/hoanmyTran/dissimilarity_deepfake_detection.git
```

注意：

- `dissimilarity_deepfake_detection` 只用于查看 SwiGLU 和门控实现；
- 不要把第二个项目的环境整体安装进 SLS 环境；
- 不要直接复制 MultiConv、CKA 和 Hugging Face XLS-R 部分（实现非常麻烦）。

三、建立修改分支并备份

进入 SLS 项目：

```
cd SLSforASVspoof-2021-DF  
  
git checkout -b hybrid-gating  
  
cp model.py model_baseline.py  
cp main.py main_baseline.py  
cp model.py model_hybrid.py
```

Windows CMD：

```
copy model.py model_baseline.py  
copy main.py main_baseline.py  
copy model.py model_hybrid.py
```

最终只修改：

```
model_hybrid.py  
main.py
```

原 `model.py` 和 `model_baseline.py` 都保留。

四、先验证原项目（已完成，跳过）

按照官方环境安装。官方仓库要求 Python 3.7、PyTorch 1.12.1，并安装其固定 fairseq 版本。
([GitHub](#))

```
conda create -n SLS python=3.7  
conda activate SLS  
  
pip install torch==1.12.1 torchvision==0.13.1 torchaudio==0.12.1  
  
unzip fairseq-a54021305d6b3c4c5959ac9395135f63202db8f1.zip  
  
cd fairseq-a54021305d6b3c4c5959ac9395135f63202db8f1  
pip install --editable ./  
cd ..  
  
pip install -r requirements.txt
```

先检查：

```
python -m py_compile model.py main.py
```

如果已有数据和权重，优先先跑一次原模型评估。官方训练命令是：([GitHub](#))

```
CUDA_VISIBLE_DEVICES=0 python main.py \  
  --track=DF \  
  --lr=0.000001 \  
  --batch_size=5 \  
  --loss=WCE \  
  --num_epochs=50
```

五、先确认原项目真实张量维度

这是最重要的一步。

打开原 `model.py`，找到 XLS-R 输出多层特征的位置。在完成 `layer_results` 堆叠后临时加入：

```
print("layer feature shape:", feature.shape)
raise SystemExit
```

或者如果变量叫 `hidden_states`：

```
print("hidden_states shape:", hidden_states.shape)
raise SystemExit
```

运行一个 batch，确认最终是不是：

```
[B, 24, T, 1024]
```

如果是：

```
[24, B, T, 1024]
```

则需要先转换：

```
hidden_states = hidden_states.permute(1, 0, 2, 3)
```

后续方案统一假设输入多层特征为：

```
[B, L, T, D]
```

其中：

- (B): batch size;
- (L=24): XLS-R层数;
- (T): 时间帧数;
- (D=1024): 特征维度。

确认后删掉：

```
raise SystemExit
```

六、在 model_hybrid.py 中加入三个模块

在文件开头确认已有：

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

然后加入以下三个类。

1. 统计感知SLS

```
class StatisticalSLS(nn.Module):
    """
    输入:
        x: [B, L, T, D]

    输出:
        fused: [B, T, D]
        weights: [B, L]
    """

    def __init__(
        self,
        feature_dim=1024,
        hidden_dim=128,
        use_std=True,
        dropout=0.1,
    ):
        super(StatisticalSLS, self).__init__()

        self.use_std = use_std

        stat_dim = feature_dim * 2 if use_std else feature_dim

        self.weight_predictor = nn.Sequential(
            nn.Linear(stat_dim, hidden_dim),
            nn.SELU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, 1),
```

```

    )

def forward(self, x):
    # x: [B, L, T, D]

    mean = torch.mean(x, dim=2)
    # [B, L, D]

    if self.use_std:
        variance = torch.var(
            x,
            dim=2,
            unbiased=False,
        )

        std = torch.sqrt(
            torch.clamp(variance, min=1e-5)
        )

        statistics = torch.cat(
            [mean, std],
            dim=-1,
        )
        # [B, L, 2D]
    else:
        statistics = mean

    logits = self.weight_predictor(statistics)
    # [B, L, 1]

    weights = torch.sigmoid(logits)
    # 保留SLS思想，不使用Softmax

    weighted = x * weights.unsqueeze(2)
    # [B, L, T, D]

    fused = torch.sum(weighted, dim=1)
    # [B, T, D]

    return fused, weights.squeeze(-1)

```

2. SwiGLU特征门控

为了兼容旧版 PyTorch，直接使用基础算子：

```

class SwiGLUGate(nn.Module):
    """
    输入输出:
        [B, T, D]
    """

    def __init__(
        self,
        input_dim=1024,
        hidden_dim=128,
        dropout=0.1,
    ):
        super(SwiGLUGate, self).__init__()

        self.norm = nn.LayerNorm(input_dim)

        self.gate_proj = nn.Linear(
            input_dim,
            hidden_dim,
        )

        self.value_proj = nn.Linear(
            input_dim,
            hidden_dim,
        )

        self.out_proj = nn.Linear(
            hidden_dim,
            input_dim,
        )

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        residual = x

        x = self.norm(x)

        gate = F.silu(
            self.gate_proj(x)
        )

        value = self.value_proj(x)

        x = gate * value
        x = self.out_proj(x)

```

```
x = self.dropout(x)

return residual + x
```

这里采用的是：

```
[
\operatorname{SiLU}(W_{gx})\odot W_{vx}
]
```

如果你查看 Multi-level 官方仓库后发现作者严格采用：

```
torch.sigmoid(...)
```

也可以将：

```
gate = F.silu(self.gate_proj(x))
```

改为：

```
gate = torch.sigmoid(self.gate_proj(x))
```

但实验中所有模型必须统一使用一种实现。

3. 时间注意力池化

先采用最简单稳定版本，不直接加入复杂多头注意力：

```
class TemporalAttentionPooling(nn.Module):
    """
    输入:
        x: [B, T, D]

    输出:
        pooled: [B, D]
        weights: [B, T]
    """

    def __init__(
        self,
        input_dim=1024,
```

```

        hidden_dim=128,
        dropout=0.1,
    ):
        super(TemporalAttentionPooling, self).__init__()

        self.attention = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Tanh(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x):
        scores = self.attention(x)
        # [B, T, 1]

        weights = torch.softmax(
            scores,
            dim=1,
        )

        pooled = torch.sum(
            weights * x,
            dim=1,
        )
        # [B, D]

        return pooled, weights.squeeze(-1)

```

七、修改原 Model 类

不要重写 XLS-R 前端。

找到原 Model.__init__()，保留：

- XLS-R加载；
- checkpoint读取；
- 原始数据接口；
- 原训练流程。

在初始化函数中加入：

```

self.use_stat_sls = bool(
    int(getattr(args, "use_stat_sls", 1))
)

self.use_swiglu = bool(
    int(getattr(args, "use_swiglu", 1))
)

self.use_temp_attn = bool(
    int(getattr(args, "use_temp_attn", 1))
)

```

然后加入三个模块：

```

self.stat_sls = StatisticalSLS(
    feature_dim=1024,
    hidden_dim=128,
    use_std=self.use_stat_sls,
    dropout=0.1,
)

self.swiglu = SwiGLUGate(
    input_dim=1024,
    hidden_dim=128,
    dropout=0.1,
)

self.temporal_pool = TemporalAttentionPooling(
    input_dim=1024,
    hidden_dim=128,
    dropout=0.1,
)

self.hybrid_classifier = nn.Sequential(
    nn.LayerNorm(1024),
    nn.Linear(1024, 256),
    nn.SELU(inplace=True),
    nn.Dropout(0.2),
    nn.Linear(256, 2),
)

self.last_layer_weights = None
self.last_temporal_weights = None

```

八、修改 `forward()` 的最小方式

保留原 XLS-R 特征提取代码。

在得到多层隐藏状态后，保证其形状是：

```
[B, 24, T, 1024]
```

假设变量名为：

```
hidden_states
```

后续加入：

```
fused, layer_weights = self.stat_sls(  
    hidden_states  
)  
# fused: [B, T, 1024]
```

然后：

```
if self.use_swiglu:  
    fused = self.swiglu(fused)
```

接着：

```
if self.use_temp_attn:  
    pooled, temporal_weights = self.temporal_pool(  
        fused  
    )  
else:  
    pooled = torch.max(  
        fused,  
        dim=1,  
    )[0]  
  
    temporal_weights = None
```

最后：

```
logits = self.hybrid_classifier(pooled)
```

```
output = F.log_softmax(  
    logits,  
    dim=1,  
)
```

保存权重：

```
self.last_layer_weights = (  
    layer_weights.detach()  
)  
  
if temporal_weights is not None:  
    self.last_temporal_weights = (  
        temporal_weights.detach()  
    )  
else:  
    self.last_temporal_weights = None
```

最后保持原 `forward()` 返回格式。

如果原模型返回：

```
return output
```

就保持：

```
return output
```

如果原模型返回：

```
return output, feature
```

就保持：

```
return output, pooled
```

不要随意改变 `main.py` 中期待的返回值数量。

九、完整的核心 forward 模板

只将以下代码放在“已经得到 hidden_states”之后：

```
# hidden_states必须为 [B, L, T, D]

fused, layer_weights = self.stat_sls(
    hidden_states
)

if self.use_swiglu:
    fused = self.swiglu(fused)

if self.use_temp_attn:
    pooled, temporal_weights = self.temporal_pool(
        fused
    )
else:
    pooled = torch.max(
        fused,
        dim=1,
    )[0]
    temporal_weights = None

logits = self.hybrid_classifier(pooled)

output = F.log_softmax(
    logits,
    dim=1,
)

self.last_layer_weights = (
    layer_weights.detach()
)

if temporal_weights is not None:
    self.last_temporal_weights = (
        temporal_weights.detach()
    )
else:
    self.last_temporal_weights = None

return output
```

十、给 main.py 增加参数

找到参数定义区域，加入：

```
parser.add_argument(
    "--use_stat_sls",
    type=int,
    default=1,
    choices=[0, 1],
)

parser.add_argument(
    "--use_swiglu",
    type=int,
    default=1,
    choices=[0, 1],
)

parser.add_argument(
    "--use_temp_attn",
    type=int,
    default=1,
    choices=[0, 1],
)

parser.add_argument(
    "--model_type",
    type=str,
    default="hybrid",
    choices=["baseline", "hybrid"],
)
```

十一、在 `main.py` 中切换模型

不要直接固定导入。

把：

```
from model import Model
```

替换为：

```
from model_baseline import Model as BaselineModel
```

```
from model_hybrid import Model as HybridModel
```

模型实例化处改为：

```
if args.model_type == "baseline":  
    model = BaselineModel(args, device)  
else:  
    model = HybridModel(args, device)
```

如果原项目实例化方式不是：

```
Model(args, device)
```

而是：

```
Model(device)
```

则要保持原接口，例如：

```
model = HybridModel(device)
```

这一步以原 main.py 为准，不要强改参数形式。

十二、先做模块级测试

新建：

```
test_hybrid_modules.py
```

内容：

```
import torch  
  
from model_hybrid import (  
    StatisticalSLS,  
    SwiGLUGate,  
    TemporalAttentionPooling,  
)
```

```

def main():
    batch_size = 2
    num_layers = 24
    time_steps = 201
    feature_dim = 1024

    x = torch.randn(
        batch_size,
        num_layers,
        time_steps,
        feature_dim,
    )

    sls = StatisticalSLS(
        feature_dim=feature_dim,
        hidden_dim=128,
        use_std=True,
    )

    fused, layer_weights = sls(x)

    assert fused.shape == (
        batch_size,
        time_steps,
        feature_dim,
    )

    assert layer_weights.shape == (
        batch_size,
        num_layers,
    )

    swiglu = SwiGLUGate(
        input_dim=feature_dim,
        hidden_dim=128,
    )

    refined = swiglu(fused)

    assert refined.shape == fused.shape

    pooling = TemporalAttentionPooling(
        input_dim=feature_dim,
        hidden_dim=128,
    )

```

```

pooled, temporal_weights = pooling(refined)

assert pooled.shape == (
    batch_size,
    feature_dim,
)

assert temporal_weights.shape == (
    batch_size,
    time_steps,
)

weight_sum = temporal_weights.sum(dim=1)

assert torch.allclose(
    weight_sum,
    torch.ones_like(weight_sum),
    atol=1e-5,
)

print("All module tests passed.")

if __name__ == "__main__":
    main()

```

运行：

```

python -m py_compile model_hybrid.py
python test_hybrid_modules.py

```

应输出：

```

All module tests passed.

```

十三、真实单batch测试

在正式训练前，必须测试：

1. 前向传播

在训练代码中临时加入：

```
batch = next(iter(train_loader))

audio = batch[0].to(device)

model.eval()

with torch.no_grad():
    output = model(audio)

print("output shape:", output.shape)
raise SystemExit
```

预期：

```
[B, 2]
```

2. 反向传播

删除前面的测试，再临时加入：

```
model.train()

output = model(audio)

loss = criterion(
    output,
    labels.to(device),
)

loss.backward()

print("loss:", loss.item())

for name, parameter in model.named_parameters():
    if (
        parameter.requires_grad
        and parameter.grad is None
    ):
        print("No gradient:", name)

raise SystemExit
```

主要确认以下模块都有梯度：

```
stat_sls  
swiglu  
temporal_pool  
hybrid_classifier
```

十四、小数据过拟合测试

正式跑完整数据之前，抽取32~64条训练数据。

训练10~20轮，确认：

- loss明显下降；
- 训练准确率能升高；
- 没有NaN；
- 没有梯度爆炸；
- 显存稳定。

若小数据都无法拟合，不要继续完整训练。

十五、四组必要实验

最终至少跑四组。

实验0：原始基线

```
CUDA_VISIBLE_DEVICES=0 python main.py \  
  --model_type=baseline \  
  --track=DF \  
  --lr=0.000001 \  
  --batch_size=5 \  
  --loss=WCE \  
  --num_epochs=50
```

实验1：Mean+Std SLS

```
CUDA_VISIBLE_DEVICES=0 python main.py \  
  --model_type=hybrid \  
  --track=DF \  
  --lr=0.000001 \  
  --batch_size=5 \  
  --loss=WCE \  
  --num_epochs=50 \  
  --use_stat_sls=1 \  
  --use_swiglu=0 \  
  --use_temp_attn=0
```

实验2：Mean+Std SLS + SwiGLU

```
CUDA_VISIBLE_DEVICES=0 python main.py \  
  --model_type=hybrid \  
  --track=DF \  
  --lr=0.000001 \  
  --batch_size=5 \  
  --loss=WCE \  
  --num_epochs=50 \  
  --use_stat_sls=1 \  
  --use_swiglu=1 \  
  --use_temp_attn=0
```

实验3：完整模型

```
CUDA_VISIBLE_DEVICES=0 python main.py \  
  --model_type=hybrid \  
  --track=DF \  
  --lr=0.000001 \  
  --batch_size=5 \  
  --loss=WCE \  
  --num_epochs=50 \  
  --use_stat_sls=1 \  
  --use_swiglu=1 \  
  --use_temp_attn=1
```

十六、1快速运行策略

不要直接四组各跑50轮。

第一轮

全部只跑：

```
--num_epochs=3
```

检查：

- 能否运行；
- 每轮耗时；
- 显存；
- loss下降；
- 验证结果。

第二轮

选择表现最好的两组：

```
--num_epochs=15
```

第三轮

只跑最佳完整模型：

```
--num_epochs=50
```

如果算力不够：

```
--batch_size=1
```

并将两个隐藏维度从128降低为64：

```
hidden_dim=64
```

新增模块本身显存开销不大，主要显存仍来自 XLS-R 300M 和24层隐藏状态。

十七、评估方法

官方项目会先生成分数文件，再用评估脚本计算 EER。([GitHub](#))

DF评估

```
CUDA_VISIBLE_DEVICES=0 python main.py \  
  --model_type=hybrid \  
  --track=DF \  
  --is_eval \  
  --eval \  
  --model_path=/path/to/best_model.pth \  
  --  
protocols_path=database/ASVspoof_DF_cm_protocols/ASVspoof2021_DF.cm.eval.trl.t  
xt \  
  --database_path=/path/to/ASVspoof2021_DF_eval/ \  
  --eval_output=scores/scores_hybrid_DF.txt \  
  --use_stat_sls=1 \  
  --use_swiglu=1 \  
  --use_temp_attn=1
```

计算EER：

```
python evaluate_2021_DF.py \  
  scores/scores_hybrid_DF.txt \  
  ./keys \  
  eval
```

LA评估

```
python evaluate_2021_LA.py \  
  scores/scores_hybrid_LA.txt \  
  ./keys \  
  eval
```

In-the-Wild评估

```
python evaluate_in_the_wild.py \  
  scores/scores_hybrid_Wild.txt \  
  ./keys \  
  eval
```

十八、结果表格

最终至少整理为：

模型	Mean+Std SLS	SwiGLU	时间注意力	DF EER	LA EER	ITW EER
XLS-R+SLS	×	×	×			
Stat-SLS	✓	×	×			
Stat-SLS+SwiGLU	✓	✓	×			
完整模型	✓	✓	✓			

同时记录：

- 参数量；
- 单轮训练时间；
- 收敛轮数；
- 最大显存；
- 三次运行平均EER，如时间允许。

十九、可视化权重

推理后可以访问：

```
layer_weights = (  
    model.last_layer_weights[0]  
    .cpu()  
    .numpy()  
)  
  
temporal_weights = (  
    model.last_temporal_weights[0]  
    .cpu()  
    .numpy()  
)
```

绘制层权重：

```

import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 4))
plt.bar(
    np.arange(len(layer_weights)),
    layer_weights,
)
plt.xlabel("XLS-R Layer")
plt.ylabel("Weight")
plt.title("Statistical SLS Layer Weights")
plt.tight_layout()
plt.savefig(
    "layer_weights.png",
    dpi=200,
)
plt.close()

```

绘制时间权重：

```

plt.figure(figsize=(10, 4))
plt.plot(temporal_weights)
plt.xlabel("Frame")
plt.ylabel("Attention Weight")
plt.title("Temporal Attention Distribution")
plt.tight_layout()
plt.savefig(
    "temporal_weights.png",
    dpi=200,
)
plt.close()

```

建议分别选择：

- 3条真实音频；
- 3条伪造音频；
- 1条模型判断错误的音频。

二十、创新点最终表述

可以把模型命名为：

中文名称：

统计感知的层级—特征—时间三级门控网络

论文中可以写成三个创新点：

1. 统计感知敏感层选择

原 SLS 仅依据时间均值生成层权重，本文同时使用均值和标准差：

$$\begin{bmatrix} s_l = \\ [\mu(h_l), \sigma(h_l)] \end{bmatrix}$$

$$\begin{bmatrix} \alpha_l = \\ \sigma(MLP(s_l)) \end{bmatrix}$$

从而同时描述隐藏层特征的中心趋势和时序波动。

2. 细粒度特征门控

在层级融合后引入 SwiGLU：

$$\begin{bmatrix} \tilde{H} \\ H + \\ \text{SiLU}(W_o \text{left}) \\ \odot \\ W_v H \\ \text{right} \end{bmatrix}$$

进一步筛选层内与伪造检测相关的特征维度。

3. 时间注意力聚合

通过：

$$\begin{bmatrix} \beta_t = \\ \text{Softmax} \\ \text{left} \\ w^\top \tanh(WH_t) \\ \text{right} \end{bmatrix}$$

生成时间权重，并进行加权聚合，使模型关注局部声码器异常、拼接边界和不自然时序区域。

二十一、最终执行顺序

第1天

- 克隆两个仓库；
- 配置原 SLS 环境；
- 原模型单batch运行；
- 确认 XLS-R 多层特征形状。

第2天

- 加入 StatisticalSLS；
- 完成模块测试；
- 完成真实batch前向和反向。

第3天

- 加入 SwiGLU；
- 小数据过拟合。

第4天

- 加入时间注意力；
- 小数据过拟合。

第5天

- 四组模型各跑3轮。

第6~7天

- 最好的两组各跑15~20轮。

第8天

- 最佳模型完整训练或继续训练；
- DF评估。

第9天

- LA、ITW评估；
- 层权重和时间权重可视化。

第10天

- 消融表；
- 方法图；
- 创新点；
- 实验分析。

最终结论

最保险的执行方式是：

原SLS项目

- └─ 保留fairseq XLS-R
- └─ 保留RawBoost
- └─ 保留数据加载
- └─ 保留训练和评估代码
- └─ 仅替换SLS之后的三个轻量模块

不要整体迁移 Multi-level SSL 项目。拉取它只是为了核对 SwiGLU 和门控逻辑。真正落地时，使用上面这套兼容 PyTorch 1.12.1 的轻量实现，逐模块测试、逐模块消融。